

E08 · W-경로 — 문맥을 가중치로접기

이 권은 본문 ch16(Generative Adapter), ch17(Do Language Models Need Sleep?), ch18(Nested Learning 과 Memory Caching) 을 수식과 증명 없이 다시설명한다. 앞 권 이 문맥을 모델 밖에 글자로 적어 두는 계보였다면, 이 권은 같은 문맥을 모델 안 가중치 모양 으로 접어 넣고 원 문맥토큰을 지우는 계보다.

이 권은 두 개의 판정을 향해 간다. 첫째, 산출물의 모양으로 경로를 나누면 틀린다 — 첫 논문은 LoRA 와 똑같이 생긴 물건을 만들지만 그것을 만드는 시계는 chunk 이고 규칙에는 gradient 가 없다. 둘째, 갱신주파수 연속체는 표기의 다리이지 기제의 다리가 아니다 — 축 하나로 attention 부터 pre-training 까지를 이었다고 주장한 논문이 있는데, 그 축위에서 실제로 이동한 상태가 하나도 없다. 두 판정 모두 서빙 결론이 걸려있다 — 롤백 단위, 배치 가능성, 상각의 성립 여부가 여기서 갈린다.

1. KV cache 를 비우기전에

이 권의 모든 논문이 손대는 층은 하나다.

기호 풀이 W (fast weights) = 요청을 처리하는 동안만 살아 있는 고정크기 상태 텐서. 자리는 KV cache 와 같지만 성질이 반대다 — KV cache 는토큰마다 append 되어 길 이에 비례해 자라고 무손실이지만, W 는 크기가처음부터 정해져 있어 새 것을 받으면 옛 것 위에 겹쳐 쓴다. 그래서 손실 압축이다. Θ (slow weights) = 배포되는 가중치, .safetensors 로 굽는 그것. E (external store) = 모델 밖의 텍스트 · 벡터 · 그래프 저장소.

W 에 쓰는 표준형은 한 줄이다.

$$W_t = \alpha_t W_{t-1} + S_t, \quad S_t = \beta_t S_{t-1} - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t) \quad (\text{U-W})$$

기호 풀이 t = 토큰 (또는 chunk) 첨자. α_t = retention gate. 남기는 비율이며, 1 이면 아무것도 잊지 않고 0 이면 직전 상태를통째로 버린다. S_t = momentum 항. 최근 갱신 방향을 얼마나 이어갈지를 담는 버퍼이고 β_t 가 그 감쇠율이다. η_t = 쓰기보폭. k_t, v_t = 이 토큰이 만든 key-value 벡터, 즉 “무엇을 무엇에 연결해 저장할 것인가”의 쌍. ℓ = inner loss, 지금저장하려는 쌍을 W 가 얼마나 못 맞히는지의 값. $\nabla_W \ell$ = 그값을 줄이는 방향을 W 의 좌표마다 계산한 것.

이 식이 말하는 것은 두 단계라는 것뿐이다. 옛 상태에 1 보다작거나 같은 수를 곱해 남길 만큼만 남기고, 새로 들어온 쌍을 그 위에더한다. 이 권의 논문들이 갈리는 지점은 전부 (i) 더하는 항을 무엇으로만드는가와 (ii) 이 갱신을 언제 돌리는가, 둘뿐이다.

기호 풀이 “학습된다” 는 이 권에서 언제나 gradient 갱신을 뜻한다 — loss 를정의하고, 그 값을 줄이는 방향을 좌표마다 계산해서, 반대 방향으로 한 걸음옮기는 절차다. 이 절차가 없으면 그 논문에는 학습이 없다. (U-W) 에는 ∇ 가 있지만, 이 권의 세 논문 중 둘은 그 자리를 gradient 가 아닌것으로 채운다.

비유 KV cache 는 회의 녹취록 전문이고 W 는 한 장짜리 요약 카드다. 카드는회의가 길어져도 두꺼워지지 않는다. — **비유가 깨지는 곳**: 사람은요약 카드를 쓸 때 무엇을 뺐는지 알고, 필요하면 녹취록을 다시 펼친다. W 는 고정 크기 텐서 위에 겹쳐 쓰는 것이라 무엇이 지워졌는지모델도 시스템도 알지 못하고, 지운 것을 되돌리는 연산이 기제 안에 없다. 이 권의 설계에서 원 녹취록은 이미 버려져 있다.

자세한 것은 ch11 § 11.2, ch16 § 16.3.

2. 접는다는 것이 정확히무엇인가

Generative Adapter 가 이 권의 첫 논문이고, 이 라인의 조작을 가장 깨끗하게보여 준다.

핵심 한 번의 forward 로 문맥을 파라미터화하는 기제는 세 조건을 동시에만족한다. (i) 문맥을 소비하는 계산이 base 모델의 **forward pass** 하나와 파라미터 없는 후처리만으로 끝난다. (ii) 산출물이 가중치와같은 모양의 텐서다. (iii) 읽기 시점에 원 문맥 토큰이 입력에없다. 셋 중 하나라도 어긋나면 다른 기제다 — backward pass가있으면 학습이고, 산출물이 토큰이면 압축이며, 읽기에 원 문맥이 남아있으면 prompting 이다.

배치는 이렇다. base LM(7B, 동결) 이 1,024 토큰짜리 chunk 에 대해 이미 만든 hidden state 를 받아, 학습된 사영 행렬 넷이 그것을 $1,024 \times 1,024$ 정사각 상태에 누적하고, 그 상태에서 매 chunk 가중치 delta 를 다시 계산한다. 없이는 자리는 블록당 하나, attention 의 출력 투영 행렬이다.

여기서 세 가지를 짚어야 한다.

gradient 가 없다. 쓰기에 loss 도 학습률도 backward pass 도 없다. 더해지는 항은 chunk 안 hidden state 들의 자기 자신에 대한 곱, 즉 Hebbian 쓰기다. (U-W) 의 어휘로 옮기면 (k_t, v_t) 쌍이 (h, h) 로붕괴한다 (h 는 hidden state 벡터다) — key 와 value 가 분리되어 있지않고, 모델이 각 hidden state 를 자기 자신에 대해 쓴다.

아무것도 잊히지 않는다. β_t 가 항등적으로 1 이고 α_t 에 해당하는 것이 아예 없다. 감쇠도 게이팅도 축출도 없이스트림 전체가 하나의 합으로 누적된다. 그리고 합이므로 **순서가사라진다** — 문맥 chunk 를 섞어 넣어도 같은 delta 가 나온다. 이 순열불변성은 논문이 진술하지 않고 갱신식에서 곧바로 따라 나오는 사실이다.

산출물이 LoRA 다. 안정화를 위해 정사각 상태의 특이값을 전부 1 로되돌리는 정규화가 들어가는데, 그것을 rank $r = 128$ 에서 자르면 곱이 얇은행렬 두 개가 된다. 논문 자신이 그 유도를 적고 LoRA 와 동일시한다. 문맥하나가 32M floats 짜리 delta 로 나온다.

기호 풀이 $r = \text{rank}$. 큰 행렬을 얇은 두 행렬의 곱으로 근사할 때 그 얇은 쪽의폭이며, r 이 작을수록 담기는 양도 저장 바이트도 준다. LoRA 는 원가중치를 얼리고 이 얇은 두 행렬만 두는 방식이다 (→ ch03 § 03.2).

비유 이 쓰기는 필름 한 장에 사진을 여러 번 겹쳐 찍는 다중노출이다. 장수가늘어도 필름은 한 장이다. — **비유가 깨지는 곳**: 다중노출에서는나중에 찍힌 것이 더 진하게 남지만, $\beta_t = 1$ 인 이 쓰기에는최근성이 없다. 첫 chunk 와 마지막 chunk 가 정확히 같은 무게로더해진다.

자세한 것은 ch16 § 16.3.

3. 모양은 Θ 인데 시계는 chunk 다

산출물만 보면 이것은 Θ -경로다 — 만들어 내는 물건이 LoRA 이고, 크기도 Θ -경로의 delta 아티팩트 공식으로 그대로 계산된다. 그런데 시계는 chunk 이고 쓰기 규칙에는 gradient 가 없으며, 배포 체크포인트는 문맥화 전후로 **바이트 단위로 같다**.

핵심 어떤 절차를 경로에 배정할 때 두 축을 독립으로 둔다. **산출물의 모양** (텍스트 레코 드인가, fast weight 텐서인가, 가중치 delta 인가) 은 저장 · 전송 · **롤백의 단위**를 정한 다. **갱신의 시계** (토큰 · chunk 인가, 세션인가, 학습라운드인가) 는 갱신식과 **비용 구조**를 정한다. 경로 판정은 시계와 갱신식에만 맡긴다. 모양으로 나누면 이 논문이 Θ -경로로 잘못들어오고, 같은 오류가 뒤의 두 논문도 잘못 배정한다.

실익은 추상적이지 않다. Θ -경로였다면 잘못 쓴 내용을 되돌리는 방법은 이전 체크포인트로의 롤백뿐이다. 이 기제에서는 상태를 0 으로 초기화하는 것이 전부이고, 비용은 문맥을 다시 접는 forward 한 번이다. 배포 아티팩트가 불변이므로 사용자별 delta 가 오염되어도 base 모델은 감사대상이 아니다.

시스템 모델링 관점 독자의 1 번 질문에 논문이 직접 답한다 — **문맥화 이후 decode 비용은 base 모델의 decode 비용이다**. MSC 에서 질의당 0.505 TFLOPS 대 0.505 TFLOPS 로 소수 셋째 자리까지 같고, 문맥길이에 대해서도 평탄하다. 네 가지가 따라 나온다. 첫째, 그 등식이 성립하려면 **delta 가 merge 된 형태로 서빙되어야 하는데 논문은 그 조건을 명시하지 않는다**. 분리된 delta 로 서빙하면 토큰당 약 67 MFLOPs 가 더 붙고, 이는 7B forward 의 토큰당 약 14 GFLOPs 에 대해 약 0.5%다 (이 산술은 본문이 논문 제원 위에 얹은 것이다). 덧셈 항 자체는 어느 형태든 작다 — **문제는 크기가 아니라 형태 선택이 서빙 구조를 정한다는 것이다**. 둘째, **merge 하면 shared-weight batching 이 죽는다**. 사용자마다 7B 사본이 필요해진다. 분리하면 요청별 LoRA 로 배치되지만, 손으로 적합한 $r=16$ LoRA 의 수 M floats 이 아니라 **요청당 32M floats** 다. 논문은 동시 다중 사용자 서빙을 한 문장도 다루지 않는다. 셋째, **상태량이 문맥 길이와 무관하다**. 다만 교차점이 이르다 — 논문의 두 점을 이르면 약 600 토큰 아래에서는 KV cache 쪽이 더 작고, 그 아래에서 이 기제는 저장을 늘린다. 넷째, **문맥화는 학습이 아니라 prefill 이다**. 절감은 첫 pass 가 아니라 그 뒤의 모든 pass 에 있다. 그 위에 얹히는 chunk 마다 · layer 마다의 rank-128 randomized SVD 는 prefill 대비 작지만, **GEMM 이 아니고 배치가 잘 되지 않는 커널**이 깨끗한 prefill 파이프라인 한가운데에 들어온다.

자세한 것은 ch16 § 16.4, § 16.7.

4. 그런데 접는 시각이 없다

판별식을 낸다.

핵심 판별식. (1) **시계** — 상태를 바꾸는 연산이 질의 도착전에 실행되는가. (2) **예산** — 그 연산이 $B_s > 0$ 을소비하는가. (3) **대상** — $\Theta \cdot W \cdot E$ 중 어느 한 층의 상태를실제로 바꾸는가. (4) **소비자** — 바뀐 상태가 그것을 유발한질의가 아니라 **이후 질의**에 쓰이는가. **기호 풀이.** B_s = sleep 계산 예산 (FLOPs 또는 tokens). 질의가오기 전에 미리 태우는 양이다.

Generative Adapter 는 조건 (1) 에서 떨어진다. 접기가 문맥이 도착한 그자리에서 wake pass 안에 일어나고, 모델은 방금 접은 그 문맥에 대해답한다. 이 논문은 W -경로의 **wake 쪽 끝점**이다. 사실 이논문에는 “sleep” 도 “offline” 도 “idle” 도 “latency” 도 한 번 나오지 않는다.

품질 쪽도 정직하게 적는다. 가장 센 baseline 은 문맥을 그대로 프롬프트에넣는 in-context prompting 인데, Mistral 에서 Generative Adapter 가 그것을이기는 칸은 짧은 문맥 쪽 **열넷 중 셋**뿐이고 32K 에서는 32.0 대 39.3 으로 진다. 개인화 과제에서는 전체 문맥 baseline 이 F1 66.0 에 질의당 2.059 TFLOPS 인데 이쪽은 40.2 에 0.505 TFLOPS 다— 계산은 $4\times$ 줄고 **전체 문맥 품질의 61%**를 유지한다. 25.8 점 F1 결손이 그 절감의 가격이다.

초록의 헤드라인은 자기 본문 표와 화해하지 않는다. “19.5 에서 31.5 로, 63.5% 개선” 이라고 쓰는데 **31.5 는 논문 어디에도 없고** (해당 표의값은 32.0), 19.5 $\rightarrow$ 31.5 는 63.5%가 아니라 61.5%다.

한계 그럼에도 이 논문에 대해 단정할 수 있는 문장이 하나 있다 — **문맥화 이후 질의당 추론 FLOPs 가 base 모델과 같다고 보고되었다.** 단정할 수 없는 것은 “효율적이다” 이다. sleep 쪽 비용 B_s 가 수치로인쇄되지 않았고 (그림의 눈금 없는 로그축 곡선으로만 존재한다), 지연은 한번도 측정되지 않았으며, 32M floats 는 dtype 이 없어 바이트로 환산되지않는다. 비율의 두 항이 모두 비어 있는 상태에서 비율이 주장된다.

자세한 것은 ch16 § 16.4 – § 16.6.

5. 접는 시각을만든다

Do Language Models Need Sleep?(arXiv:2605.26099) 은앞 논문이 남긴 구멍을 정확히 메운다. **접는 기제는 있었고 접는시각이 없었다.** 이 논문은 접기를 답 생성 경로에서 떼어 내 **KV cache** 를 비우는 순간에 붙이고, 그 자리에서 같은 chunk 를 여러 번 다시통과시킨다.

무대를 만드는 제약이 둘이다. 첫째, **hard eviction** — chunk 크기 C 토큰마다 KV cache 를 완전히 비운다 (실험에서 C 는 $24 \cdot 75 \cdot 512 \cdot 2000$ 이다). 경계를 넘어살아남는 것은 W 뿐이므로, 경계 이후의 정확도는 전적으로 W 에무엇이 접혔는지의 함수가 된다. 둘째, **prediction-phase latency 제약** — 답 토큰 하나는 표준 forward pass 정확히 한 번으로 만든다. 추가 loop 도 chain-of-thought 토큰도 금지이고, 데이터에서 CoT 가 제거되어 있어우회로가 없다. 이 두 제약 아래에서 비교군과 처리군의 wake 경로가**커널 수준에서 동일**하다.

핵심 KV cache 를 비우기 전 **offline recurrence** 로 **fast weight** 에 접는기제란, eviction 경계에서 KV cache 를 버리기 직전, 같은 chunk 의토큰에 대해 블록 스택을 N 번 더 통과시켜 각 블록의 W 만 갱신하고, 그 과정에서 정제된 은닉 특징과 재구축된

KV cache 는 경계에서 폐기하여 오직 W 만 경계를 넘게 하는 절차다. 세 요소가 모두 필요하다 — 경계 직전이라는 시각, 같은 토큰의 반복 횟수, W 이외 전부의 폐기. 특징을 함께 넘기면 depth-recurrent 예측 모델이고, 경계 이후에 돌리면 문맥 재적재이며, 한 번만 돌리면 통상의 SSM 이다.

기호 풀이. N = sleep pass 수, 즉 같은 chunk 를 몇 번 다시접는가. run 마다 고정된 하이퍼파라미터이고, N 값마다 별개로 학습된 모델이다. halting 기제도 예제별 배분도 없다.

의사코드가 지고 있고 산문이 말하지 않는 사실이 셋이다. W 는 pass 사이에 초기화되지 않는다 — pass p 는 pass $p-1$ 이 끝난 상태에서 시작하므로 같은 토큰들이 누적으로 N 번 접힌다. 은닉 특징은 chunk 안에서 pass 를 건너 운반된다 — 그래서 pass 마다 저장되는 쌍 (k_t, v_t) 가 달라진다. 그리고 경계에서 파괴된다. 따라서 chunk 를 넘는 유일한 채널이 W 다.

그러므로 N 을 늘리는 것은 “(U-W) step 을 더 밟는 것” 이 아니다. 상태 $\rightarrow$ 특징 $\rightarrow$ 게이트 $\rightarrow$ 상태의 닫힌 고리를 입력 토큰을 올려 둔 채 N 번 도는 것이며, 늘어나는 것은 저장량이 아니라 버려진 문맥위에서 수행 가능한 계산의 깊이다. 이 논문의 진단 전환이 정확히 그것이다 — 하이브리드가 긴 문맥에서 지는 이유를 선행 연구는 상태 용량 부족으로 보았는데, 이 논문은 답기까지의 계산량 부족으로 옮긴다. 저장할 정보량을 고정한 채 추론 깊이만 올리면 baseline 이 무너지는 것이 그증거다.

비유 시험 전날 같은 노트를 여러 번 다시 읽는 것과 같다. 노트의 분량은 그대로인데 읽을수록 머릿속에 남는 구조가 깊어진다. — 비유가 깨지는 곳: 사람이 다시 읽는 대상은 매번 같은 원문이다. 여기서는 pass 마다 읽는 대상이 직전 pass 의 상태가 새로 만들어낸 특징이고, 원 토큰의 embedding 은 첫 pass 뒤로 다시 등장하지 않는다. 그리고 경계에서 노트도 원문도 전부 태우고 머릿속만 남긴다.

자세한 것은 ch17 § 17.2 – § 17.3.

6. N 은 무엇을 사는가

세 무대가 있고 전부 합성이며 전부 저자가 만들었거나 다시 파라미터를 잡았다. 확립된 긴 문맥 벤치마크는 기각의 대상으로만 등장하고, 자연텍스트 위의 결과가 없다.

첫 무대인 Rule-110 자동자에서 N 을 올리면 정확도가 10%대에서 30%대로 오른다. 문맥 길이도 eviction 규칙도 예측 시점 계산도 고정되어 있으므로 개선이 consolidation 시점의 추가 계산에서 왔다는 귀속은 성립한다. 그러나 과제는 여전히 풀리지 않았다 — 거의 5B 학습토큰 뒤의 최고치가 30%대다. 그리고 초록이 “transformer 도 하이브리드도 실패한다” 고 쓰는 것은 결과가 아니라 동어반복이다. hard eviction 아래에서 표준 transformer 는 예측 전에 KV cache 가 파괴되므로 무작위 추측을 넘을 수 없다. 캐시를 유지하도록 허용한 full-attention baseline 은 어느 과제, 어느 스케일에서도 실행되지 않았다.

수치가 나오는 무대는 GSM-Infinite 하나다.

표 E08-1. GSM-Infinite 정확도. 절대 차는 백분점이며 본문이 논문 수치에서 계산한 값이다.

설정	모델	연산 수	loop 없음	최대 N	절대 차	논문 라벨
hard eviction	Jet-Nemotron 2B	6	0.742	0.812	+7.0	9%
hard eviction	Jet-Nemotron 2B	8	0.351	0.388	+3.7	11%
hard eviction	Ouro 1.4B	6	0.419	0.615	+19.6	47%
hard eviction	Ouro 1.4B	8	0.210	0.272	+6.2	30%
sliding window	Ouro 1.4B	2	0.596	0.905	+30.9	52%
sliding window	Ouro 1.4B	8	0.116	0.137	+2.1	18%

표 옆에 놓아야 할 사실이 넷이다.

이득이 어려운 꼬리에만 있다. 쉬운 문제에서는 N 이 아무것도사지 않는다 — 2 연산과 4 연산에서 N 전 구간의 폭이 0.3 에서 2.9 백분점이다. 논문의 라벨이 전부 **상대** 개선이라 표에서 큰 수로보이는 것이 절대치로는 작아지는 자리도 있다.

N 에 대한 비단조가 있다. 가장 어려운 8 연산에서 $N = 2$ 가 $N = 1$ 보다 근소하게 낮다 (0.209 대 0.210). 논문은 이 반전을 논의하지 않고, 초록의주장은 “sleep 깊이를 늘리면 성능이 오른다” 이다.

가장 큰 헤드라인은 결손의 회복이다. 52%는 0.596 에서 0.905 로 가는 값인데, **같은 모델**이 다른 eviction 설정에서 같은 2 연산을 loop 없이 0.857 – 0.868 로 푼다. 새 능력이 아니라 설정이만든 구멍을 메운 값이다.

계산량을 맞춘 baseline 이 없다. loop 는 비용을 N 에 대략비례해 늘리는데 모든 비교가 학습 토큰 또는 **step** 을 맞춘것이지 FLOP 을 맞춘 것이 아니다 — 같은 x 좌표에서 $N = 4$ 실행은 이미 4 배계산을 썼다. “재귀가 통상의 계산으로는 살 수 없는 것을 산다” 는 중심주장이 더 넓거나 더 깊거나 더 오래 학습한 **비-loop 모델**과 겨뤄진적이 없다. 여기에 시드가 하나이고 learning rate 는 $N = 1$ 에서만 튜닝되었다는 사실이 겹친다.

한계 비용 네 칸이 전부 비어 있다. B_s 는 구조로만 진술되고 추론시점의 sleep 비용이 어떤 형태로도 보고되지 않는다 (유일한 계산량 수치는 run 당 1-2 H100 GPU-day 라는 학습 예산이다). C_{cap} 은 상태 차원 · head 수 · dtype 이없어 유도조차 불가능하다. ρ 는 관측될 레짐에 논문이 들어가지않는다 — 요청당 경계가 1-6 개이고 W 가 sequence 마다 0 으로초기화되므로 반복 열화를 켈 대상이 만들어지지 않는다. 그리고 L_w 가가장 중요하다. “wake 지연을 보존한다” 는 헤드라인은 **프로토콜구성상 참이지 측정이 아니다**. 답 토큰마다 표준 forward pass 한 번으로 못박아 두었으니 wake 경로가 baseline 과 같을 수밖에 없다. ms/token 도 TTFT 도 없고, 논문의 유일한 throughput 그림은 명시적으로 **훈련** 측정이다.

자세한 것은 ch17 § 17.5 – § 17.6.

7. 판별식은 통과하는데 상각은 되지않는다

이 논문은 W -경로가 sleep-time compute 이 될 수 있음을 보인**유일한 정면 사례**다. 다만 그 통과를 지고 있는 것이 헤드라인실험이 아니다.

Depo 설정에서는 네 조건이 전부 선다. 문맥이 먼저 전부 흘러가고 그 뒤에질의 열 개가 붙으므로 한 번 접은 W 를 **열 개의 질의가공유한다**. 반면 헤드라인인 GSM-Infinite 에서는

논문이 질문을 문맥앞에 놓는다. 그러면 consolidation 이 질의 조건부가 되어 조건 (1) 이 문자 그대로는 성립하지 않고, 요청당 질의가 하나이므로 조건 (4) 도약해진다.

그 약화의 대가가 정확히 상각의 소멸이다.

$$C_{\text{avg}} = C_{\text{wake}} + \frac{C_{\text{sleep}}}{N_q} \quad (\text{A})$$

기호 풀이 N_q = 한 문맥을 공유하는 질의 수. C_{sleep} = 그 문맥에한 번 지불한 sleep 비용. C_{wake} = 질의 하나를 처리하는비용.

이 식이 말하는 것은 미리 태운 계산이 질의 수로 나뉜다는것뿐이다. Depo에서는 $N_q = 10$ 이 구조로 확정되어 있다. GSM-Infinite에서는 $N_q = 1$ 이므로 식이 $C_{\text{avg}} = C_{\text{wake}} + C_{\text{sleep}}$ 으로 붕괴한다. **상각이 아니라 재배치다** — wake에서 뺀 계산이 같은 요청 안의다른 국면으로 옮겨 갈 뿐이고, 요청 전체의 계산량은 N 에 대략 비례해늘어난다.

시스템 모델링 관점 서빙 관점의 결론이 다섯이다. 첫째, **decode**에서는 아무것도 바뀌지않는다. 토큰당 decode FLOPs 도, 가중치 트래픽도, 산술 강도도 $N = 1$ 실행과 같다. 달라지는 것은 W 의 내용뿐이다. 둘째, 대신 **eviction 경계마다 chunked prefill 버스트가 붙는다**. 논문제원으로 어렵하면 블록 절반만 도는 설정에서 $N = 6$ 일 때 한 chunk 의 consolidation 비용이 평범한 prefill 의 3.5 배이고 요청당 경계가 1-6 개다 (이 곱셈은 본문의 산술이고 논문은 하지 않는다). 셋째, **그버스트 안에 성질이 반대인 두 커널이 있다**. 고정 크기 fast weight 는 $N \times C$ 번 read-modify-write 되는 **대역폭 가벼운 · 계산 무거운** 작업이고, 같은 loop 안의 attention 은 pass 마다 KV cache 를 다시 쌓으므로 chunk 에 대한 제공 비용이 N 번반복된다. 넷째, **eviction 경계는 요청별 토큰 수에서 발생하므로배치 안에서 정렬되지 않는다**. 한 요청이 자고 나머지가 decode 하는배치에는 커널 모양이 둘 살아 있다. 스케줄러는 자는 요청의 이웃을세우든지 consolidation 을 별도 잡으로 빼든지 해야 하고, 논문은 이 경계 skew 를 다루지 않는다. 다섯째, **상태가 요청보다 오래 살지 않아**사용자당 지속 저장소가 생기지 않는다 — checkpoint/restore 도 세션 캐시 축출 정책도 요청 간 상각도없다. 그리고 서빙 결정에 필요한 비교 팔이 데이터에서 제거되어 있다 — “경계에서 N 번의 sleep pass” 와 “decode에서 CoT 토큰 몇 개” 가 같은품질을 얼마에 사는지가 SLA 를 가르는 질문인데, 논문은 CoT 를 빼는 것으로그 팔을 없앴다.

한계 그러므로 판별식을 통과하는 것과 상각되는 것은 같은 일이 아니다. E -경로의 sleep 산출물은 요청보다 오래 살아 N_q 가 1 보다 커질 수있다. 이 논문의 sleep 산출물은 요청과 함께 죽는다. 같은 이름 아래 있지만회계의 종류가 다르다.

자세한 것은 ch17 § 17.4, § 17.7.

8. 축 하나로 전부 이을 수 있는가

앞 두 절의 조작 — 같은 규칙을 더 자주, 또는 다른 시각에 적용하는 것 — 을보면 자연스러운 일반화가 떠오른다. **갱신 주기를 축으로 놓으면 attention 부터 pre-training 까지 한 줄에 놓이는 것 아닌가.** Nested Learning(arXiv:2512.24695) 이 정확히 그것을 주장한다.

기호 풀이 f = 갱신 주파수. 데이터 한 점을 처리하는 시간을 단위로 삼아, 어떤성분이 단위 시간에 몇 번 갱신되는지를 센 값이다. attention 의 KV 읽기때 토큰이므로 $f = \infty$ 자리에, pre-training 으로 굳은 MLP 는 문맥안에서 한 번도 안 움직이므로 $f = 0$ 자리에 놓인다. 논문은 그 사이를 채우고, 학습과 추론의 구분 자체가 지식 전달 과정을 최고 주파수 level 에서 최저 주파수 level (즉 pre-training) 로부터 끊어낸 결과라고 쓴다.

이 주장이 성립하면 이 책의 3 층 프레임은 축 하나로 접힌다. 그래서 검증한다.

핵심 연속체 주장이 **기제의 다리** 이려면 두 조건이 필요하다. (i) 축 위의 어떤 갱신이 **문맥 경계를 넘어 존속**하고, (ii) 그 산출물이 **배포 아티팩트**에 되돌아 쓰인다. 둘 중 하나라도 없으면 그 연속체는 **표기의 다리**다 — 축 하나가 두 끝점을 이름으로 이을뿐, 어떤 상태도 끝점 사이를 실제로 이동하지 않는다.

논문의 움직이는 객체를 전부 나열하면 결과가 이렇다. **문맥이 끝난뒤에도 남는 행이 하나도 없다.** 논문 자신이 자기 범주를 그렇게 정의한다 — 획득된 문맥 내 지식은 현재 문맥이 제거되면 존속하지 않는다고 박스로 진술한다. 배포 아티팩트를 바꾸는 행은 딱 하나, 15B 토큰 continual pre-training 인데 그것은 제안 모델에도 비교군에도 **똑같이** 적용된다. 두 팔에 고정된 값은 기여로 셀 수 없다. 그리고 실측된 가장 느린 눈금이 **2K 토큰**이다 — 세션도 sleep 라운드도 축 위에 인스턴스가 없다.

결정적 증거는 논문 자신의 문장이다. 신경과학 유비를 세우면서 빠른 온라인 consolidation 과 느린 **오프라인** consolidation 을 둘 다 이름 붙여 소개한 뒤, 본 연구는 첫 단계인 온라인 과정에 집중한다고 곧바로 범위를 좁힌다. **오프라인 pass** 를 이름 붙여 부르고 명시적으로 사양한다.

비유 라디오 다이얼이다. 눈금 양 끝에 방송국 이름을 적어 두면 그 사이가 연속으로 보인다. — **비유가 깨지는 곳:** 다이얼 끝에 이름을 적는 것과 다이얼을 돌려 그 방송에 실제로 도달하는 것은 다른 사건이다. 주파수를 아무리 낮춰도 얻는 것은 **느리게 움직이는** W 이지 Θ 가 아니다. 문맥 경계를 넘는 존속과 체크포인트로의 되쓰기는 주파수를 낮춰 도달할 수 있는 상태가 아니라 **다른 종류의 연산**이다.

한계 이 판정이 논문의 기여를 깎지 않는다. 이 논문은 W 층을 주파수로 구조화하는 어휘를 만들었고 뒤의 Θ -경로 논문들이 그 어휘를 빌려 쓴다. 판정이 말하는 것은 **어휘가 건너간 자리에 기제는 건너가지 않았다**는 것이다. 그리고 다리가 아니라는 사실이 이 책의 3 경로 구분을 정당화한다 — 주파수 하나로 두 층이 이어졌다면 경로 구분은 눈금의 구분에 불과했을 것이다. 덧붙여 이 논문의 ablation 캡션은 “모든 구성요소가 긍정적으로 기여한다”고 적는데, 마지막 행이 그것을 반증한다 — level 을 만드는 사영 하나를 제거하면 perplexity 가 12.24 에서 12.19 로 내려간다.

자세한 것은 ch18 § 18.2, § 18.4, § 18.6.

9. 개수를 늘리면 — W 층의 용량천장

같은 저자 그룹의 Memory Caching(arXiv:2602.24281)은 반대편을 누른다. 앞 논문이 “ W 의 갱신 주기가 하나뿐이라 문제”라고 했다면, 이쪽은 “ W 의 개수가 하나뿐이라 문제”라고 한다. 시퀀스를 세그먼트로 자르고 각 세그먼트 끝의 W 를 버리지 않고 보존한 뒤, 읽기를 그 캐시들 위의 게이트된 혼합으로 바꾼다.

핵심 어떤 논문이 갱신식을 바꾼다는 것은 쓰기 규칙의 우변을 바꾸는 것이다. 읽기식을 바꾼다는 것은 상태를 만드는 규칙은 그대로 두고 그것을 읽는 방식만 바꾸는 것이다. 둘은 비용 회계에서 다른 방향으로 들어간다 — 갱신식 변경은 B_s 에 들어가 식 (A)로 상계될 수 있고, 읽기식 변경은 질의마다 다시 지불되므로 원리적으로 상계되지 않는다.

이 논문은 갱신식을 한 글자도 바꾸지 않는다. 세그먼트 경계의 사건은 갱신이 아니라 체크포인트 저장이고 추가로 소비되는 추론은 0이므로, $B_s = 0$ 이라 sleep-time compute 이 아니다. 그럼에도 이 권에 들어오는 이유는 이 논문이 W -경로에서 가장 비싼 사실을 공급하기 때문이다.

W 층에서 recall 을 사려면 상태가 자라야 한다. 캐시 개수의 상계를 정하는 것은 아키텍처가 아니라 세그먼트 분할 스케줄이다 — 균일분할이면 개수가 문맥 길이에 선형이라 상계가 없고, 로그 분할이면 시퀀스 길이 L 에 대해 $\log_2 L$ 개로 상계가 생기지만 먼 과거의 해상도를 내놓아야 한다. W -경로가 “고정 크기 상태”라는 이름으로 파는 이점은 recall 요구가 높아지는 순간 사라진다. 그리고 그렇게까지 해도 격차는 좁혀지되 닫히지 않는다 — 가장 어려운 recall 설정에서 Transformer 40.8 대 최고 변형 32.2 이고, 검색 평균에서도 41.00 대 40.50 이다. 이 논문의 ablation 캡션도 자기 표에 반증된다 — 한 행이 00.0 / 00.0 / 00.0 으로 비어 있는데 캡션은 모든 설계 선택이 기여한다고 단정한다.

비유 브라우저 탭을 닫지 않고 계속 열어 두는 것이다. 필요한 것을 다시 찾지 않아도 된다. — 비유가 깨지는 곳: 열어 둔 탭은 볼 때만 비용이 든다. 여기서는 **토큰마다** 열린 캐시 전부에 대해 forward 를 돌리므로, 열어 둔 개수가 그대로 매 토큰의 계산량이 된다.

시스템 모델링 관점 아래 산술은 본문이 논문 제원 위에 얹은 계산이며 논문의 보고값이 아니다. 논문이 바이트를 적지 않으므로 bf16 을 가정하고 나머지는 1.3B 설정의 제원을 쓴다. 체크포인트 하나가 약 75.5M 파라미터, 약 151 MB 다. 문맥 16K 에 세그먼트 256 이면 캐시가 64 개이므로 시퀀스당 약 9.7 GB 이고, 같은 모델의 KV cache 는 16K 에서 약 2.4 GB 이니 캐시 쪽이 약 4 배 무겁다. 손익분기 세그먼트 길이는 약 1,024 이며, 배치를 다르게 잡으면 배수는 더 커진다. 단정하는 것은 방향과 자릿수다 — 어느 배치 가정에서도 시퀀스당 상태가 같은 모델의 KV cache 보다 무겁다. 읽기 쪽이 더 날카롭다. 회수가 토큰마다 모든 캐시에 대한 forward 이므로 decode 토큰당 약 9.7 GFLOP 이고, 같은 모델의 dense forward (약 2.6 GFLOP/token) 의 약 3.7 배다. decode 가 완전히 대역폭 지배가 된다. 그래서 서빙 가능한 변형은 top- k 만 로드하는 희소 변형 하나뿐이다 — 세그먼트 요약이 사전 계산 가능하므로 router 가 캐시 상태를 읽지 않고 돌고, 캐시를 HBM 밖에 두고 선택된 것만 끌어오는 계층 구조가 성립한다. 다만 그 k 의 값이 논문에 없고, 서빙 가능한 변형과 성능이 좋은 변형이 같린다. 배치 쪽은 유리하다

— 캐시는 시퀀스별로 다르지만 Θ 는 공유되므로 shared-weight batching이 깨지지 않는다. 대신 페이지 단위가 토큰이 아니라 수백 MB 짜리 체크포인트가 되어 단편화와 스케줄링 난도가 오른다.

자세한 것은 ch18 § 18.3, § 18.5, § 18.7.

10. 침묵의 지도

Generative Adapter는 fast weight 계보를 두텁게 인용하면서 **consolidation** 쪽은 **한편도 인용하지 않는다** — EWC도 replay도 생물학적 sleep도 참고문헌에 없고, 그 문헌들은 전부 이 논문보다 앞서므로 연대가 강제된 침묵이 아니다. Memory Caching은 전문에서 “sleep”이 0회 등장하고 $E \cdot \Theta$ 두 경로 인용이 각각 0건이다. 반대로 Do LMs Need Sleep?은 두 경로를 모두 인용하고 산문으로 자기와 구별한다 — 다만 교전의 깊이가 균일하게 한 문장이고 **실험적 접촉이 0이다**. 모든 그림의 비교 집합이 자기 자신의 $N = 1$ 판본이다. 그리고 더 큰 단절이 경로 안에 있다 — 이 논문은 gradient가 아닌 국소 규칙을 입장으로 내세우는데, 그것은 W -경로의 다른 절반이 정확히 반대편을 잡은 자리다.

자세한 것은 ch16 § 16.8, ch17 § 17.8, ch18 § 18.8.

요약

- W -경로는 문맥을 고정 크기 상태에 접어 넣고 원 문맥 토큰을 버린다. KV는 자라고 무손실이며 W 는 크기가 고정이라 겹쳐 쓰고 손실적이다. 무엇이 지워졌는지 알 방법도, 되돌릴 연산도 기제 안에 없다.
- Generative Adapter는 forward 한 번으로 문맥을 파라미터화한다. gradient도 loss도 학습률도 없고, 쓰기가 hidden state의 자기 곱이라 key와 value가 붕괴하며, 아무것도 잊히지 않고, 합이므로 순서가 사라진다.
- 산출물의 모양으로 경로를 나누면 틀린다. 이 논문은 LoRA를 만들지만 시계가 chunk이고 배포 체크포인트가 불변이다. 모양은 저장·전송·롤백 단위를 정하고, 경로는 시계와 갱신식이 정한다 — 롤백이 “이전 체크포인트로 되돌리기”가 아니라 “상태를 0으로 초기화”인 것이 그 차이의 운영 쪽 얼굴이다.
- 그러나 접기가 wake 안에서 일어나 판별식 조건 (1)에서 떨어진다. 품질도 좋다 — in-context prompting을 이기는 칸이 열넷 중 셋이고, 전체 문맥품질의 61%를 유지한다 (40.2 대 66.0).
- Do LMs Need Sleep?이 접는 시각을 만든다. KV cache를 비우기 직전 같은 chunk를 N 번 더 통과시키고, 정제된 특징과 재구성된 KV는 경계에서 버려 **오직 W 만 경계를 넘게 한다**. 늘어나는 것은 저장량이 아니라 버려진 문맥 위에서 수행 가능한 계산의 깊이이다.
- 그 이득은 어려운 꼬리에만 있고 (쉬운 문제에서 N 전 구간의 폭이 0.3–2.9 백분점), N 에 대해 비단조인 자리가 있으며, 가장 큰 상대이득 52%는 같은 모델이 다른 설정에서 loop 없이 이미 도달하는 수준의 회복이다. **계산량을 맞춘 baseline이 없고** 시드가 하나이며 세무대가 전부 합성이다.

- 비용 네 칸이 전부 비어 있다. 특히 “wake 지연을 보존한다” 는 헤드라인은답 토큰당 forward 한 번으로 못 박은 **프로토콜 구성상 참이지측정이 아니다**.
- 판별식 통과와 상각은 같은 일이 아니다. W 가 sequence 마다 0 으로초기화되므로 산출물이 요청과 함께 죽고, 헤드라인 설정에서는 $N_q = 1$ 이라 식 (A) 가 붕괴한다. **상각이 아니라 요청 내부의재배치다**.
- 갱신 주파수 연속체는 표기의 다리이지 기제의 다리가 아니다. 문맥을 넘어 존속하는 행이 하나도 없고, 배포 아티팩트를 바꾸는 유일한행은 두 팔에 똑같이 적용된 통제군이며, 실측된 가장 느린 눈금이 2K 토큰이다. 결정적으로 논문 자신이 오프라인 단계를 명시적으로 사양한다.
- Memory Caching 은 갱신식이 아니라 읽기식을 바꾸고 $B_s = 0$ 이다. 읽기식 변경은 질의마다 다시 지불되므로 원리적으로상각되지 않는다. 대신 이 논문이 W 층의 가격표를 공급한다 — recall 을사려면 상태가 문맥에 선형으로 자라야 하고, 상계를 걸려면 해상도를내놓아야 하며, 그 성장의 상수는 같은 모델의 KV cache 보다 크다.
- 두 논문 모두 ablation 캡션이 자기 표에 반증된다 — 한쪽은 제거하면 perplexity 가 좋아지는 행이 있고, 다른 쪽은 00.0 / 00.0 / 00.0 으로빈 행 위에 “모든 설계 선택이 기여한다” 를 세운다.

자가 점검

- W 와 KV cache 를 성장 방식과 손실 여부로 가르고, W -경로가 원문맥 토큰을 버리는 대가로 무엇을 얻고 무엇을 잃는지 말할 수 있다.
- Generative Adapter 의 쓰기에서 $\beta_t = 1$ 이 무엇을 금지하는지셋 (감쇠 · 최근성 · 순서) 열거하고, 순열 불변성이왜 대화와 시연 과제에서 위험한지 설명할 수 있다.
- “산출물의 모양으로 나누면 틀린다” 를 롤백 단위의 차이로 설명할 수 있다.
- offline recurrence 의 세 요소 (경계 직전, 반복 흡수, W 이외 전부폐기) 를 대고, 하나씩 빼면 각각 어떤 다른 기제가 되는지 말할 수 있다.
- 어떤 설정에서 $N_q = 10$ 이고 어떤 설정에서 $N_q = 1$ 인지 말하고, 후자에서 식 (A) 가 무엇으로 붕괴하는지 쓸 수 있다.
- 연속체가 기제의 다리인지 판정하는 두 조건을 대고, 어느 행이통제군이라서 세지 못하는지 설명할 수 있다.
- 갱신식을 바꾼 논문과 읽기식을 바꾼 논문을 구분하고, 그 구분이 비용회계의 어느 항목으로 나타나는지 말할 수 있다.
- (Rosetta) 이 권의 논문들이 “모델 재배포” 행의 물건을 만들면서 “KV cache” 행의 시계로 움직인다는 것을 설명하고, 그 어긋남이 “batch 로 weight 공유” 행에서 merge 형태와 분리 형태로 각각 어떻게 나타나는지말할 수 있다.

다음 권으로

이 권에서 만들어진 것은 전부 **가중치처럼 생겼지만 요청과 함께죽는** 상태였다. 배포 아티팩트는 어느 논문에서도 바뀌지 않았고, 주파수를낮춰서는 거기에 닿지 못한다는 것이 이 권의 판정이다.

그러므로 다음 층으로 가려면 다른 종류의 쓰기 연산이 필요하다. 다음 권은 Θ 에 실제로 쓰는 primitive — LoRA 와 모델 편집 — 를 세우고 그상한을 쟁다. 옮겨 가는 질문은 셋이다. **forward 쓰기가 gradient 쓰기에 내주는 품질이 얼마인가** (이 권의 어느 논문도 같은 모양 · 같은 rank 의 gradient baseline 을 돌리지 않았다), 세그먼트 개수로 나타난 용량 천

장이 거기서는 무엇으로 나타나는가, 그리고배포 아티팩트가 바뀌는 순간 shared-weight batching 에 무슨 일이 일어나는가.